

Nombre: Ricardo Fuentes Monter Roberto Daniel Ramos Flores		Matrícula: T02938286 T02966616
Nombre del curso: Desarrollo de aplicaciones web	Nombre del profesor: Viridiana Mares Rodríguez	
Módulo: 1	Actividad: evidencia 1	
Fecha: 27/09/2026		

This activity will be completed in teams of two.

Perform the analysis and interpretation of the problem and start the project; to do this you must do the following

- Create the Github repository of the project.
- Choose and justify a work methodology that is ideal for the planning and execution of the project.
- Design the diagrams that will help you to describe graphically how the web application will behave. Among the diagrams that you should design, consider the following:
 - BPMN diagram.
 - Class diagrams
 - Activity diagrams
 - Use case diagrams
- Choose the database and its respective ER diagram containing all the entities with their attributes, and the way they are related.
- Personal reflection.

1. Create the Github repository of the project.

link: <https://github.com/RichieMonter/Evidencia-1>

2. Work Methodology (Metodología de Trabajo)

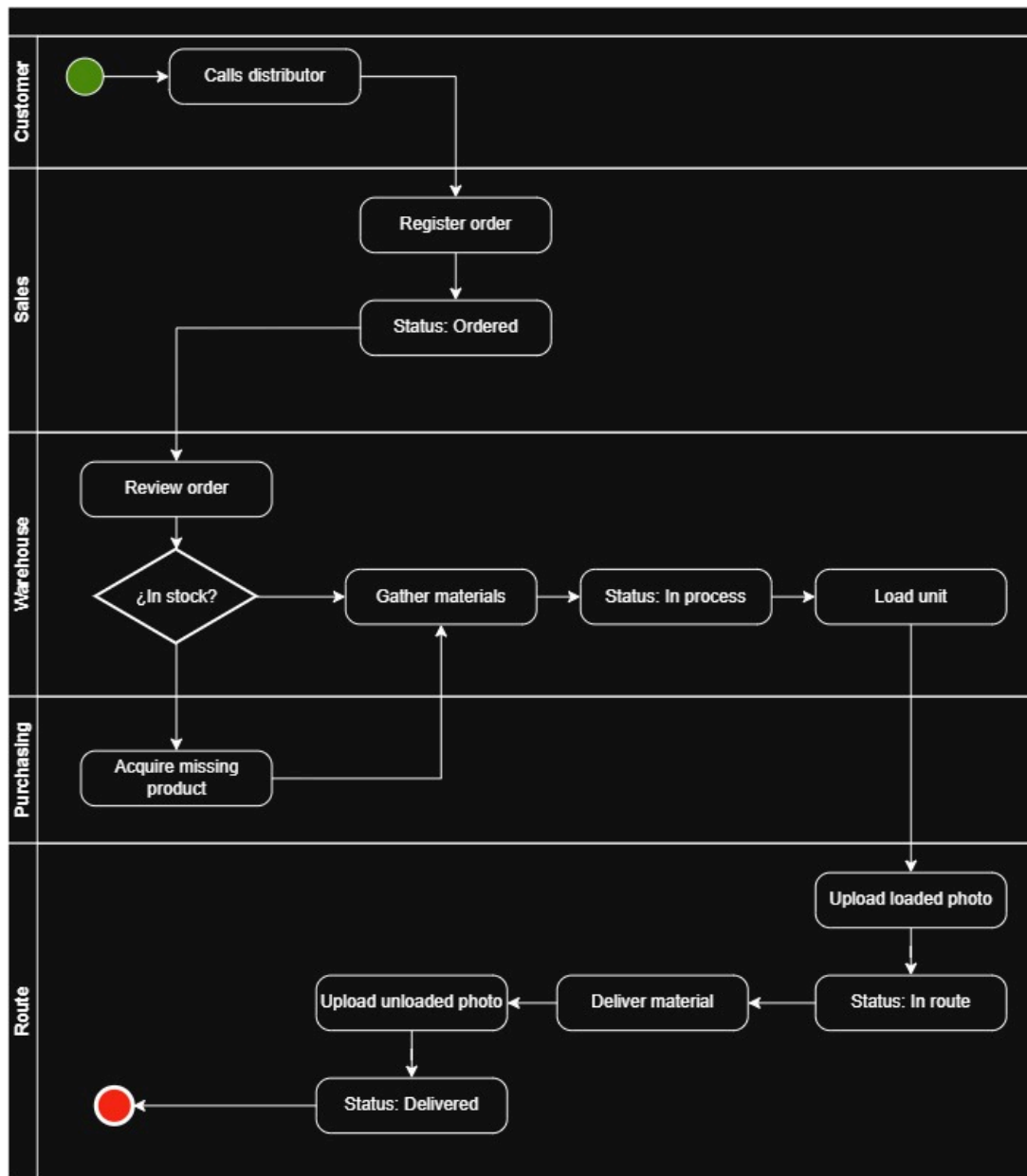
The agile methodology based on Scrum was chosen because it is ideal for developing modular web applications with clearly defined roles. Scrum allows the order lifecycle (Sales, Warehouse, Purchasing, Route) to be broken down into "User Stories" that can be developed and tested in short iterations or Sprints. This ensures that critical modules, such as customer tracking and the administrative Dashboard, can be delivered and evaluated progressively, reducing the risk of business logic errors. Additionally, it facilitates adaptation if requirements for photographic evidence or order statuses change during development.

Español:

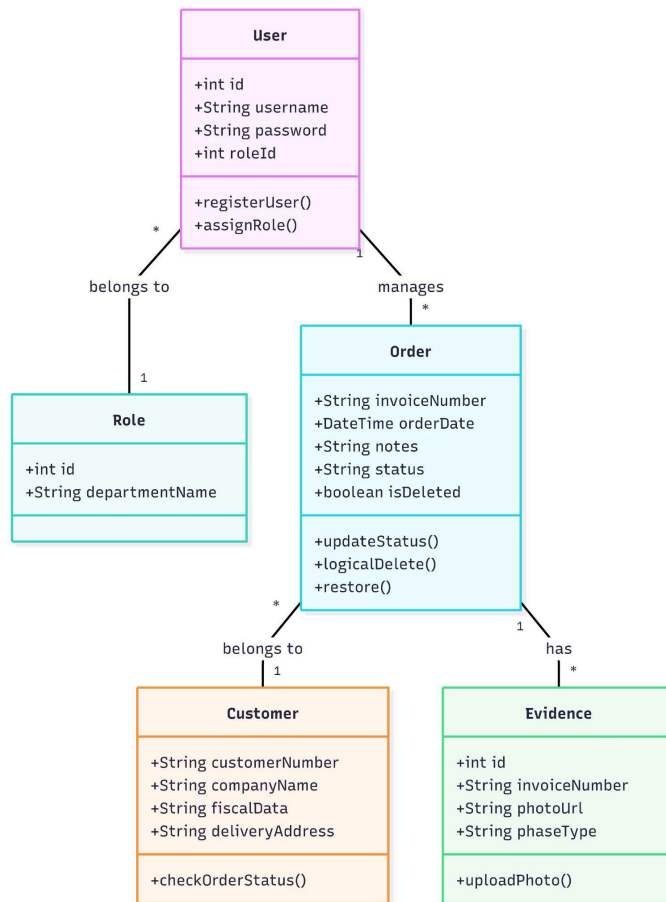
Se elige la metodología ágil basada en Scrum porque es ideal para el desarrollo de aplicaciones web modulares con roles claramente definidos. Scrum permite dividir el ciclo de vida del pedido (Ventas, Almacén, Compras, Ruta) en "Historias de Usuario" (User Stories) que se pueden desarrollar y probar en iteraciones cortas o *Sprints*. Esto garantiza que los módulos críticos, como el seguimiento para los clientes y el Dashboard administrativo, puedan ser entregados y evaluados progresivamente, reduciendo el riesgo de errores en la lógica del negocio. Además, facilita la adaptación en caso de que los requerimientos de las evidencias fotográficas o los estados del pedido cambien durante el desarrollo.

3. Diagrams

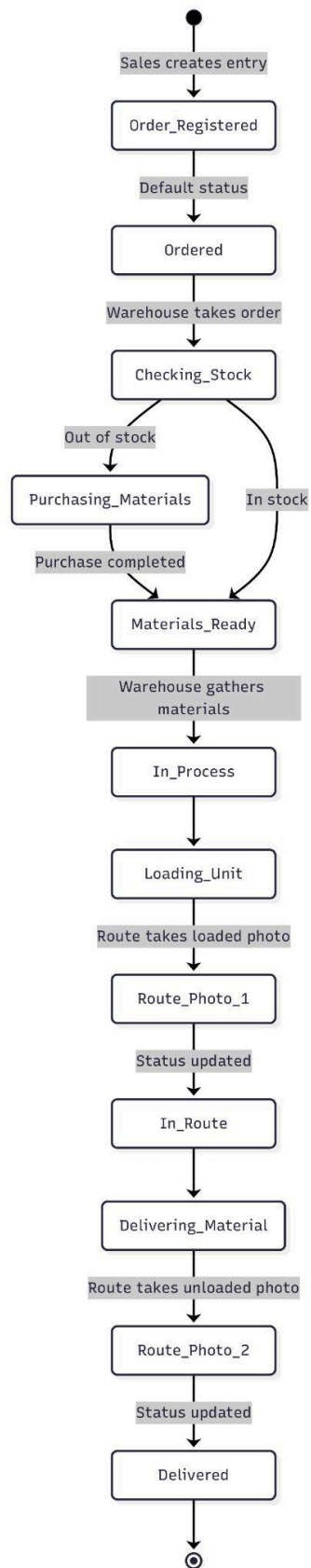
A. BPMN Diagram (Business Process)



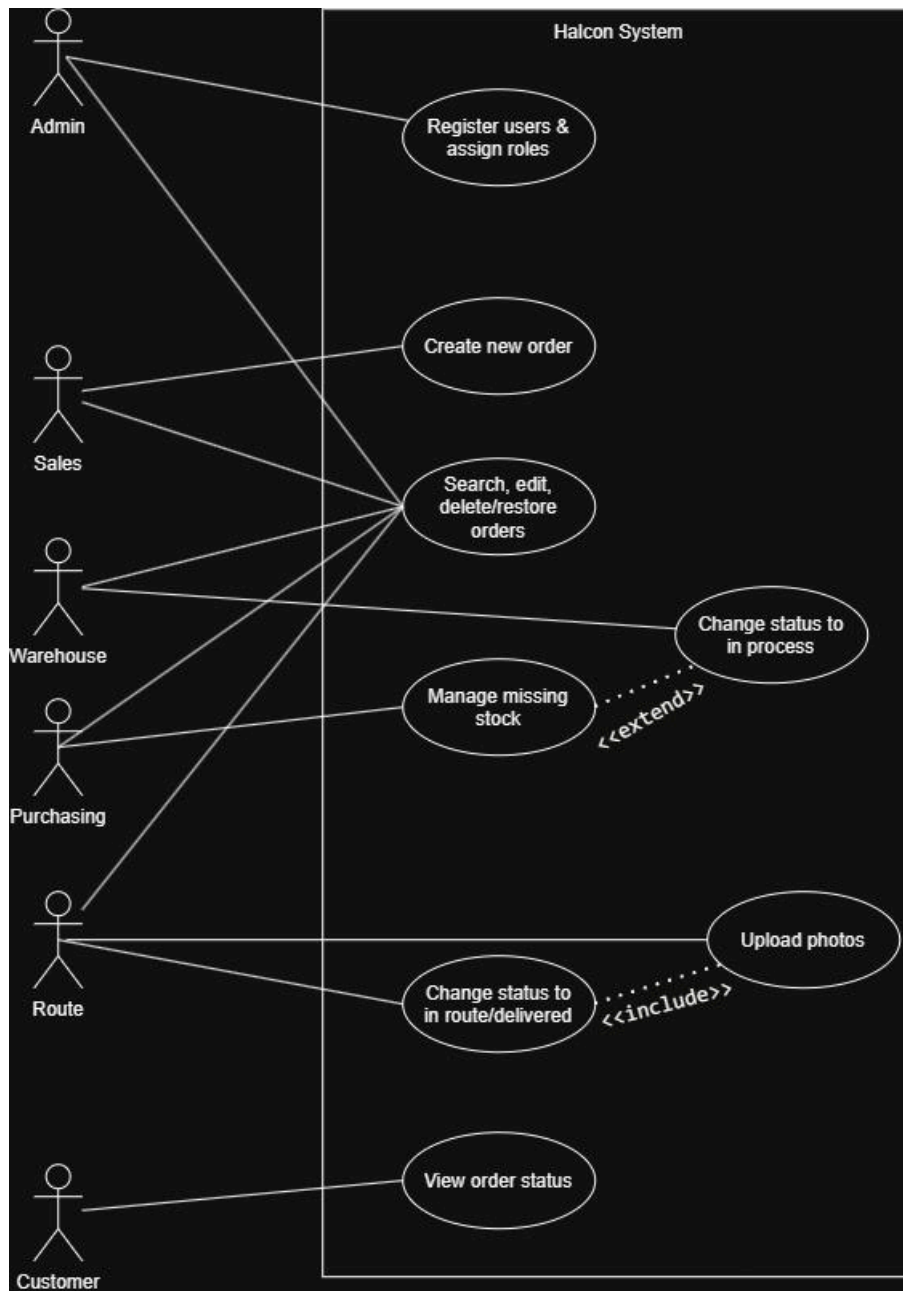
B. Class Diagram



C. Activity Diagram



D. Use Case Diagram



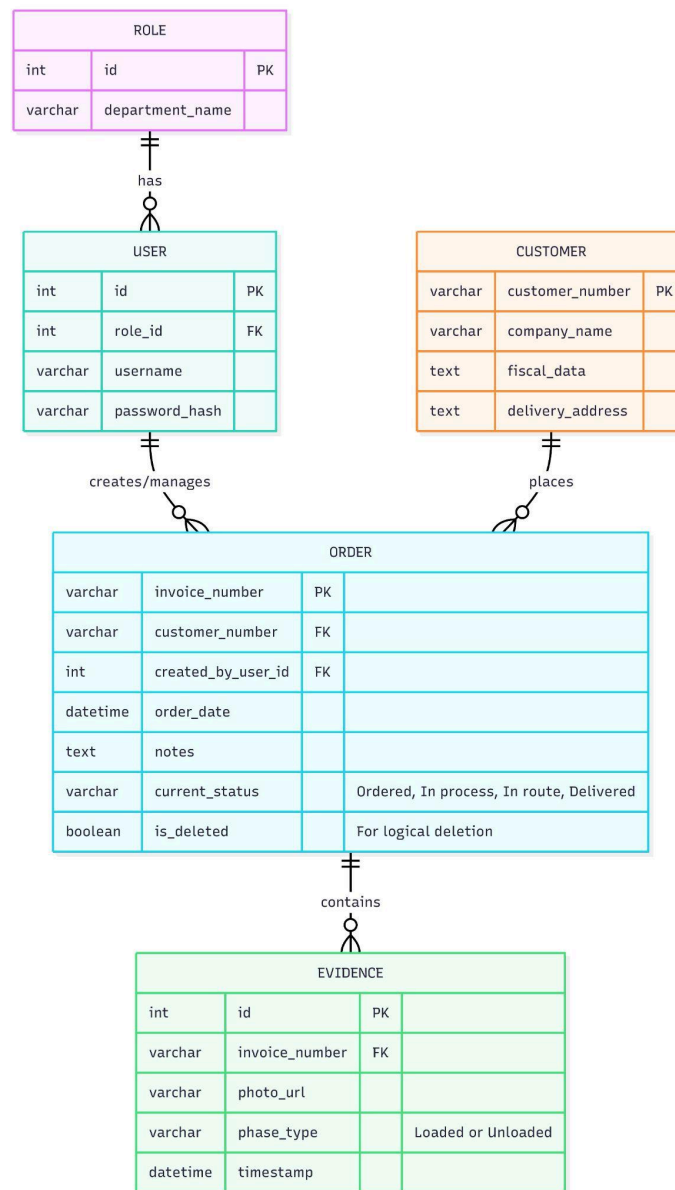
4. Base de Datos (Database y ER Diagram)

Selected Database: PostgreSQL

Justification: A relational database (SQL) was chosen because the project's information has a fixed, highly interrelated structure (Users -> Roles, Orders -> Customers, Orders -> Evidence). PostgreSQL ensures data integrity, making it ideal for a system where invoice numbers and tax data must remain completely consistent and free of discrepancies.

Base de Datos elegida: PostgreSQL Justificación: Se elige una base de datos relacional (SQL) porque la información del proyecto tiene una estructura fija y altamente relacionada

(Usuarios -> Roles, Pedidos -> Clientes, Pedidos -> Evidencias). PostgreSQL asegura la integridad de los datos, ideal para un sistema donde los números de factura y los datos fiscales no deben tener inconsistencias. ER diagram:



5. Personal Reflection (Reflexión Personal)

English Version

Question 1: In what way does the formal design of data architecture and diagramming prevent errors during the coding phase of a software project?

Ricardo: Designing the architecture through UML and BPMN diagrams establishes clear business rules before writing a single line of code. By structuring the order statuses in advance (Ordered -> In process -> In route -> Delivered), we can visualize the exact flow and identify edge cases early on, such as what happens when a product is out of stock. This prevents logic errors when we start developing the actual application.

Roberto: The formal design prevents us from making structural mistakes that are hard to fix later. By defining data relationships and visibility restrictions right from the start—like who can see or upload the route photos—we avoid database inconsistencies. This planning saves a lot of time because it prevents costly refactoring and rewriting code in the middle of the development phase.

Question 2: How does the appropriate selection of roles and the use of logical deletion (Soft Deletes) impact the security and auditing of a business application?

Ricardo: The strict separation of roles guarantees the principle of least privilege. This means we make sure clients cannot manipulate internal data, and only the right staff (like route operators) have permission to attach delivery evidence. It keeps the system secure by ensuring everyone only has access to the tools they actually need for their job.

Roberto: On the data side, the implementation of Soft Deletes (`deleted_at`) is crucial for auditing. It prevents the irreversible loss of financial and logistical information due to human or intentional errors. Instead of permanently erasing an order, it just hides it, guaranteeing complete traceability and the ability to restore records for future accounting audits.

Question 3: What did I learn while developing this activity?

Ricardo: I learned that jumping straight into coding without planning is a huge mistake. Doing the diagrams first made me realize details I hadn't even thought about, like figuring out the exact steps between the warehouse and purchasing. It's actually pretty cool to see how everything makes sense on paper before typing any code, and it definitely makes the actual programming part feel less overwhelming.

Roberto: I realized how useful it is to have clear rules set up from the start, especially so that not just anyone can mess with parts of the system they shouldn't touch. Working with soft deletes was also a great takeaway for me because it gives you peace of mind knowing that if someone deletes a record by mistake, it's not gone forever and you can just bring it back.

Versión en Español

Pregunta 1: ¿De qué manera el diseño formal de la arquitectura de datos y diagramación evita errores durante la fase de codificación en un proyecto de software?

Ricardo: Diseñar la arquitectura mediante diagramas UML y BPMN establece reglas claras de negocio antes de escribir una sola línea de código. Al estructurar previamente los estados del pedido (Ordered -> In process -> In route -> Delivered), podemos visualizar el flujo exacto e identificar casos límite desde el principio, como qué hacer cuando un producto está agotado. Esto previene errores de lógica cuando empezamos a desarrollar la aplicación real.

Roberto: El diseño formal evita que cometamos errores estructurales que son difíciles de arreglar después. Al definir las relaciones de datos y las restricciones de visibilidad desde el inicio (como quién puede ver o subir las fotos de la ruta), evitamos inconsistencias en la base de datos. Esta planificación ahorra mucho tiempo porque previene refactorizaciones costosas y tener que reescribir código a mitad del desarrollo.

Pregunta 2: ¿Cómo impacta la selección adecuada de roles y el uso de borrado lógico (Soft Deletes) en la seguridad y auditoría de una aplicación empresarial?

Ricardo: La separación estricta de roles garantiza el principio de mínimo privilegio. Esto significa que nos aseguramos de que los clientes no puedan manipular datos internos, y que solo el personal adecuado (como los operadores de ruta) tenga permiso para adjuntar evidencias de entrega. Mantiene el sistema seguro al asegurar que cada persona solo tenga acceso a las herramientas que realmente necesita para su trabajo.

Roberto: Por el lado de los datos, la implementación de Soft Deletes (borrado lógico) es crucial para la auditoría. Previene la pérdida irreversible de información financiera y logística por errores humanos o intencionados. En lugar de borrar un pedido permanentemente, solo lo oculta, garantizando la trazabilidad completa y la capacidad de restaurar registros para futuras auditorías contables.

Pregunta 3: ¿Qué aprendí al desarrollar esta actividad?

Ricardo: Aprendí que saltarse directo a programar sin planear es un error enorme. Hacer los diagramas primero me hizo darme cuenta de detalles que ni se me habían ocurrido, como definir bien los pasos entre el almacén y las compras. La verdad está muy padre ver cómo todo tiene sentido en papel antes de escribir código, y definitivamente hace que la parte de programar se sienta menos pesada.

Roberto: Me di cuenta de lo útil que es tener las reglas súper claras desde el principio, sobre todo para que no cualquiera mueva cosas en el sistema que no le tocan. Aprender sobre los soft deletes también fue un gran descubrimiento para mí, porque te da esa tranquilidad de saber que si alguien borra un registro por accidente, no se pierde para siempre y lo puedes recuperar sin problema.